

KAGGLE SIMULATION COMPETITION · ORBIT WARS

Scaling Reinforcement Learning to the Stars

Training a 200M-parameter transformer to play Orbit Wars with pure self-play reinforcement learning

Isaiah Pressman

July 2026



OVERVIEW

Mission summary

200M

parameters, one transformer

15B

environment steps of self-play

2p + 4p

both game modes, one model

~0

lines of code written by hand

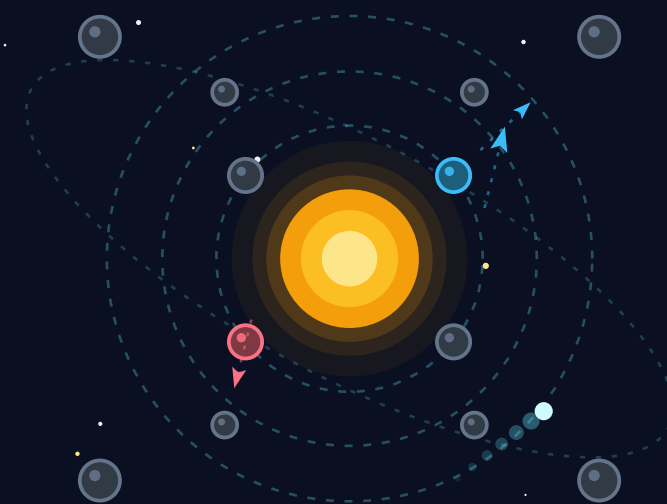
- **Pure self-play PPO** — no imitation, no human demonstrations
- **Minimal domain engineering** — raw entities in, (source, target, size) out
- **The Bitter Lesson held** — scaling beat cleverness
- **Fully agentic development** — coding agents wrote all the code

01

SECTION 01

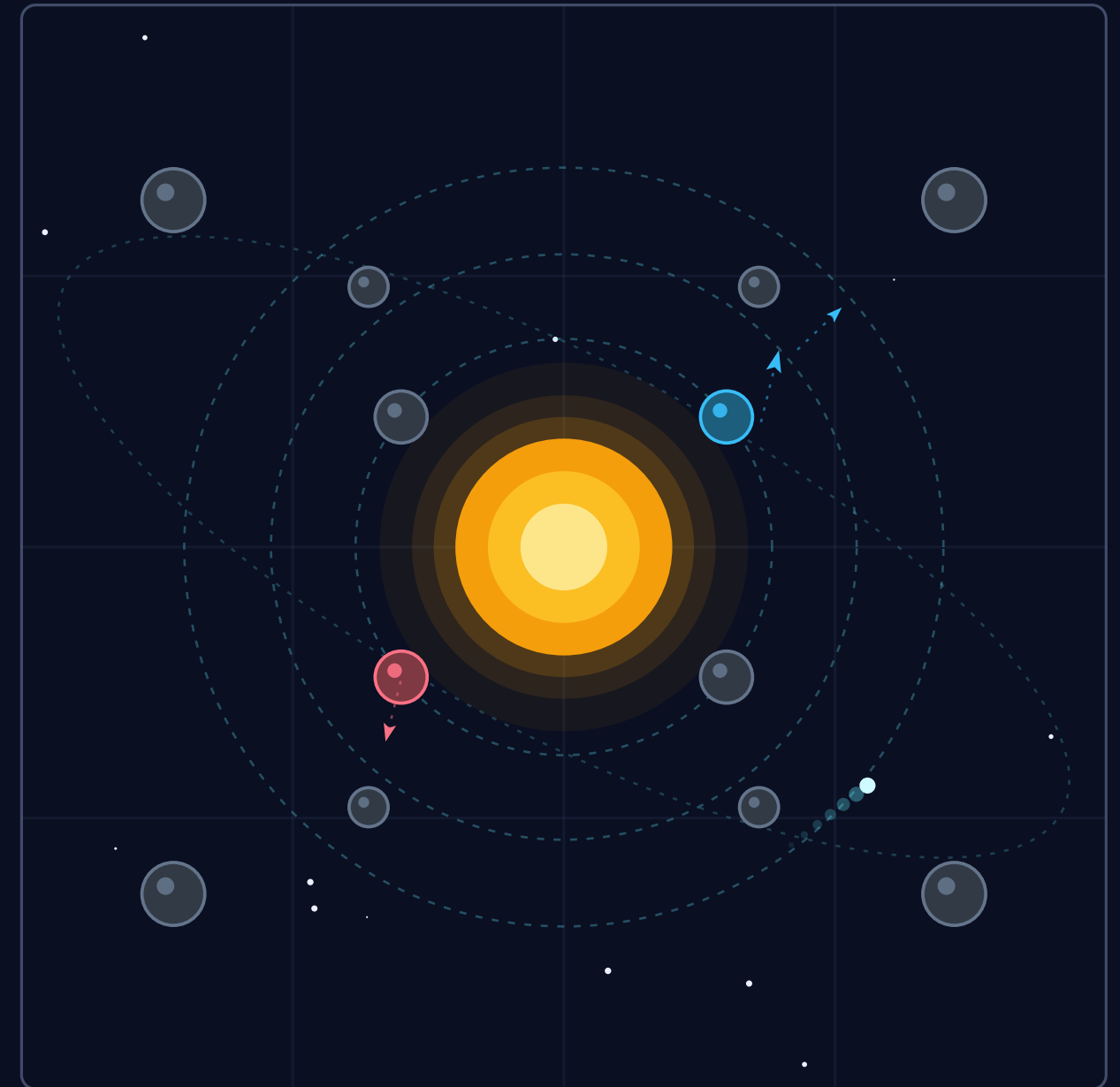
Mission briefing: the game

Orbit Wars in three slides.



A simultaneous-move strategy game in orbit

- **2 or 4 players, 500 turns** — 100×100 continuous field, all players move at once
- **The sun** destroys any fleet that crosses it
- **20–40 planets** — inner ones orbit; each produces 1–5 ships/turn
- **Comets** cross on elliptical paths — capturable, but exit with every garrisoned ship
- **4-fold mirror symmetry** — every start is fair
- **Win:** most total ships (garrisoned + in flight) at turn 500

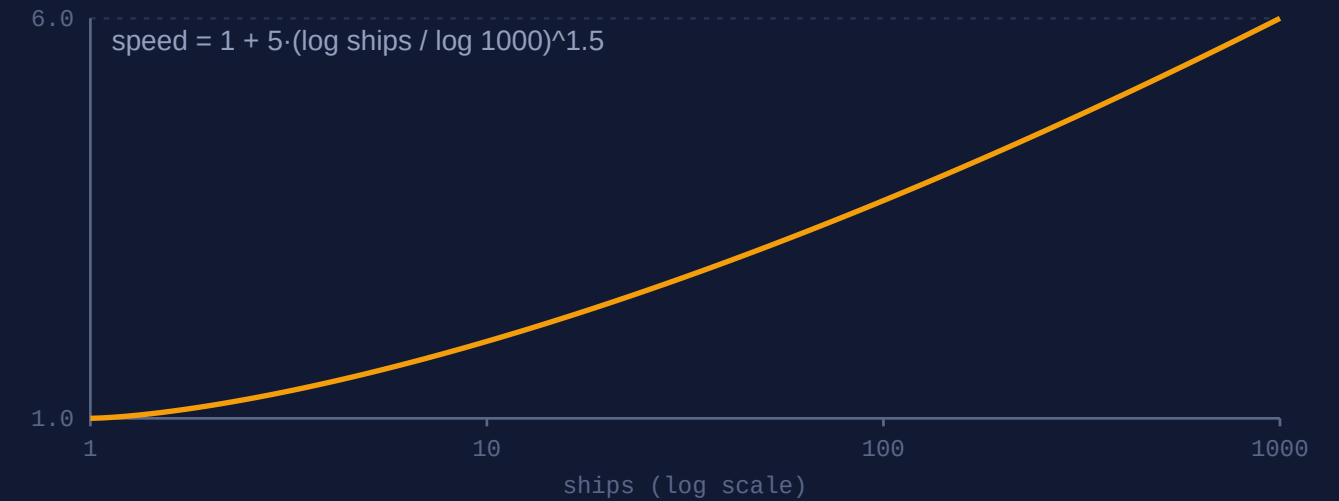


100 × 100 continuous space · 4-fold mirror symmetry

Fleets: launch, fly, fight

- **Moves:** (planet, angle, ship count) — straight lines, no steering
- **Bigger fleets fly faster** — an incentive to concentrate force
- **Fleets die** in the sun or off the board; hitting a planet → combat
- **Combat:** largest vs 2nd largest, survivor vs garrison; ties annihilate

FLEET SPEED VS SIZE



COMBAT: ONE COLLISION, RESOLVED

RED 80

BLUE 50

arrive at a neutral planet (garrison 20)

80 - 50 → RED 30

largest fights 2nd largest; difference survives

30 - 20 → RED 10

survivor fights garrison

PLANET → RED, 10 SHIPS

capture! (a tie destroys all attackers)

The agent interface

OBSERVATION — EVERY TURN

planets [id, owner, x, y, radius, ships, production]

fleets [id, owner, x, y, heading, source, ships]

comets full trajectory + position along it

extras initial planet positions, angular velocity

Raw entity lists — no pixels, no grid, no aggregation.
Everything on the board is visible to everyone.

ACTION — A LIST OF LAUNCHES

[[from_planet, angle, num_ships], ...]

Any number of launches per turn, from any owned planet.
Fleets are irrevocable once launched — no steering,
no recall, no merging mid-flight.

One decision type, made ~500 times per game,
from up to ~40 sources at once.

SUBMISSION CONSTRAINTS

1 s per turn (+60 s overage) on a slow Kaggle CPU — no GPU

100 MiB submission file, self-contained

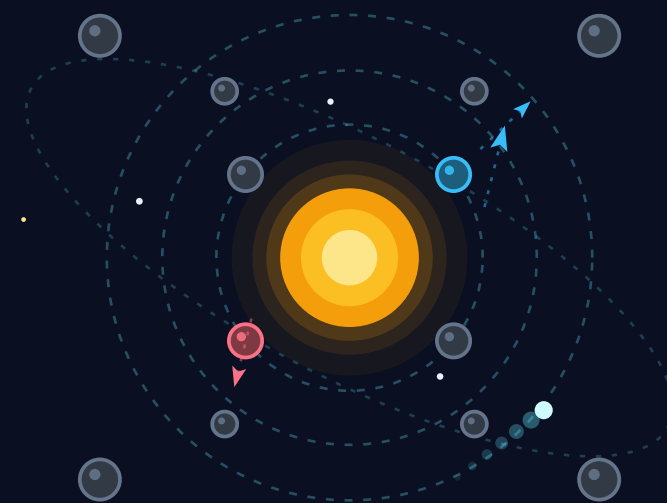
— these two limits shape the whole approach →

02

SECTION 02

Flight plan: the approach

Three self-imposed constraints, and where they led.



Three self-imposed rules

01

Fully agentic development

No code by hand. Review docs, not diffs.

02

Learn, don't engineer

Lowest-level encodings that could work. Pure self-play, no imitation.

03

Bet on the Bitter Lesson

Scale a big generic model; don't craft a clever small one.

A competition is the perfect low-stakes lab — if it fails, discard and learn.

Agentic software development

- **Agents wrote effectively all the code** — experiments running within weeks
- **Review shrank from code to docs** — checking intent, not implementation
- **Faithful to specs, light on bloat** — but creativity left much to be desired
- **Caveat: a best-case setting** — greenfield, no users, cheap mistakes

*“An incredible accelerator to the development process —
but not a substitute for thinking.”*

Observation & action encoding

- **One token per entity** — planets, comets, fleets + 5 summary tokens; no aggregation
- **Action, take 1 — raw launch angle:** semi-reasonable, far from competitive
- **Action, take 2 — pick a target planet:** competitive; the one concession
- **Tried & dropped:** planet–fleet cross-attention, discrete size bins

final interface: `entity tokens → model → (launch? · target · continuous fleet size)`

Everything else — orbits, combat math, comet timing — learned internally.

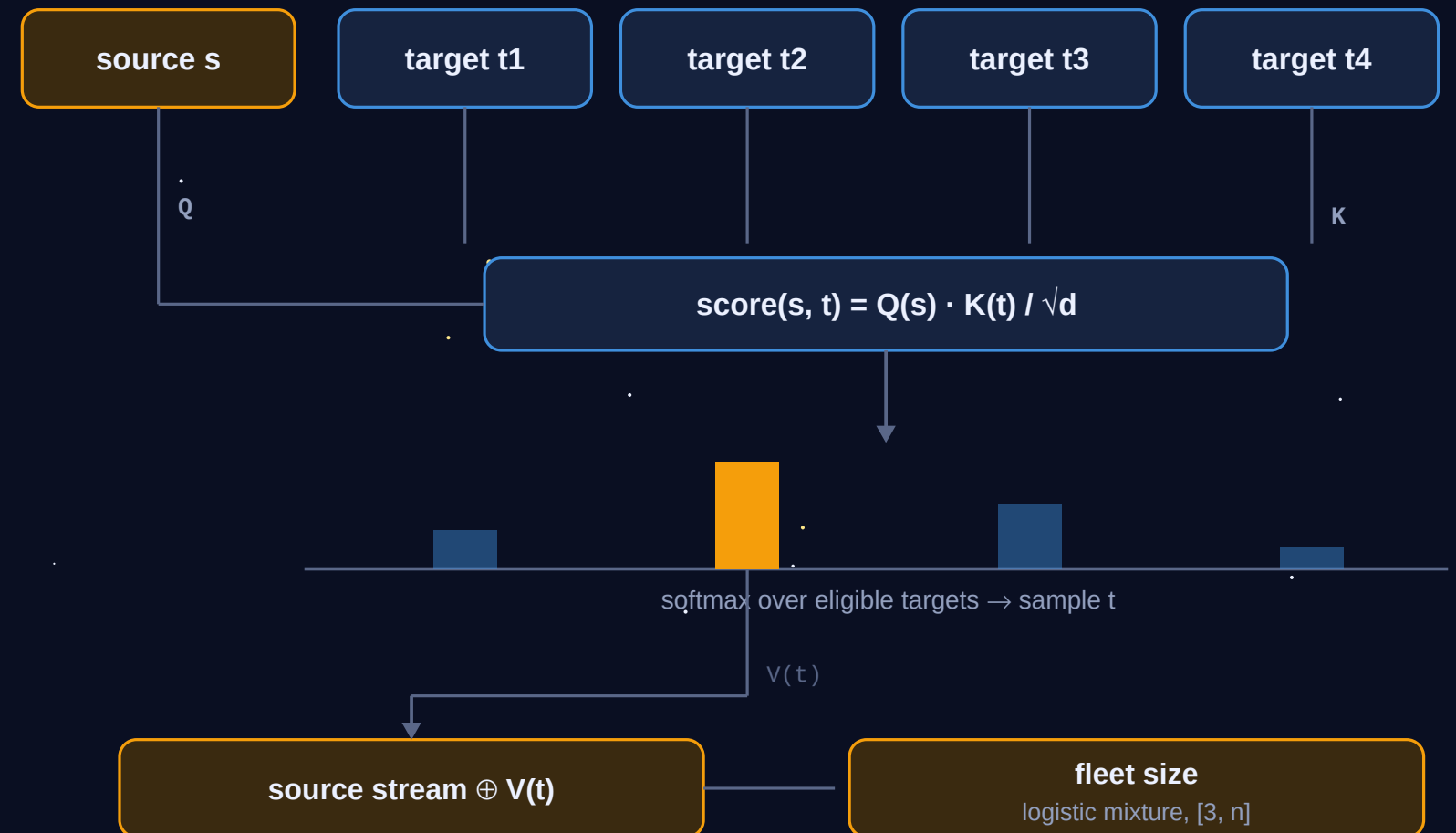
One model, every player

- **Shared 768-d token space** — 38 blocks, 16 heads, ~200M params
- **Actor:** launch? → target via $Q \cdot K / \sqrt{d}$ → mixture fleet size
- **Critic:** softmax win probability over surviving players
- **One forward pass, every player** — a 2–4× compute saving
- **17 specialty global tokens** — 5 summary, 4 plan, 4 value, 4 scratch



Target selection

- $Q(s)$ — query from the launching source token
- $K(t)$ — keys from every planet & comet token
- $\text{Sample } t \sim \text{softmax}(Q \cdot K / \sqrt{d})$ — the fleet's destination
- $V(t)$ added into the source stream — conditions the fleet-size head
- Reuses the backbone's tokens — no pointer network, no per-pair features



Training: self-play PPO

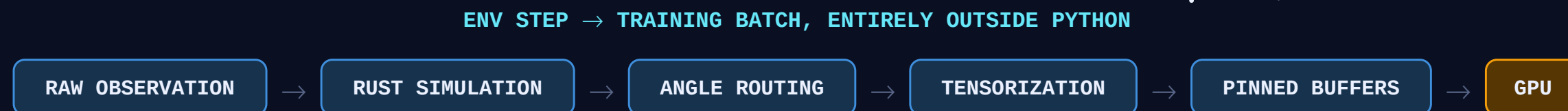
- **PPO over IMPALA** — scaling = “ask for more GPUs”; linear DDP throughput
- **Pure self-play** — 2p + 4p mixed; win +1, lose -1; $\gamma = 1.0$
- **Checkpoint gating** — replace best at >70% head-to-head; KL + value-CE anchors
- **Plus the standard toolkit** — GAE- λ , clipping, entropy bonus, warm restarts

THE ACTION-MASK SURPRISE

Masking obviously-bad actions (e.g. launches into the sun) made the model worse — hypothesis: the unmasked model must learn more of the physics. Re-enabled only for final fine-tuning + test time.

Rewriting the environment in Rust

- **Python env too slow for RL** — rewrote it in Rust, parity-tested against real replays
- **More than a simulator** — (source, target) → launch angles, routed around the sun
- **Throughput plumbing** — pinned-memory buffers, multithreaded environments
- **All agent-written** — validated by parity fixtures, not line-by-line review



200M params · 100 MiB · 1 CPU-second

SPEED: 1 S/TURN ON CPU

- **int8 linear layers** + a visible-fleet cap
- **Still too slow** in ~8% of 4p games
- **Fallback:** hot-swap to a 5M model at 1 s of overage left
- **Outcome:** the 5M model converted 100% of winning positions

SIZE: 100 MiB FILE CAP

- **4-bit NormalFloat codebook** — group 128, fp16 scales → fits the cap
- **Not too lossy:** ~40% head-to-head vs full precision
- **3-bit:** quality loss outweighed the larger model — stopped at 200M

NET EFFECT

200M parameters, 100 MiB, 1 s/turn on a slow CPU — at nearly full strength.

Scale was the strategy

Each increase in model size brought a large jump in strength — scaling stopped at the submission limits, not the learning curve.

32×B200

final training run: four 8-GPU nodes

8192

parallel rollout envs · train batch size 32,768

6.3M

env steps per GPU-hour (~110K tokens/s)

~2400

B200-hours for the final model



1M



5M



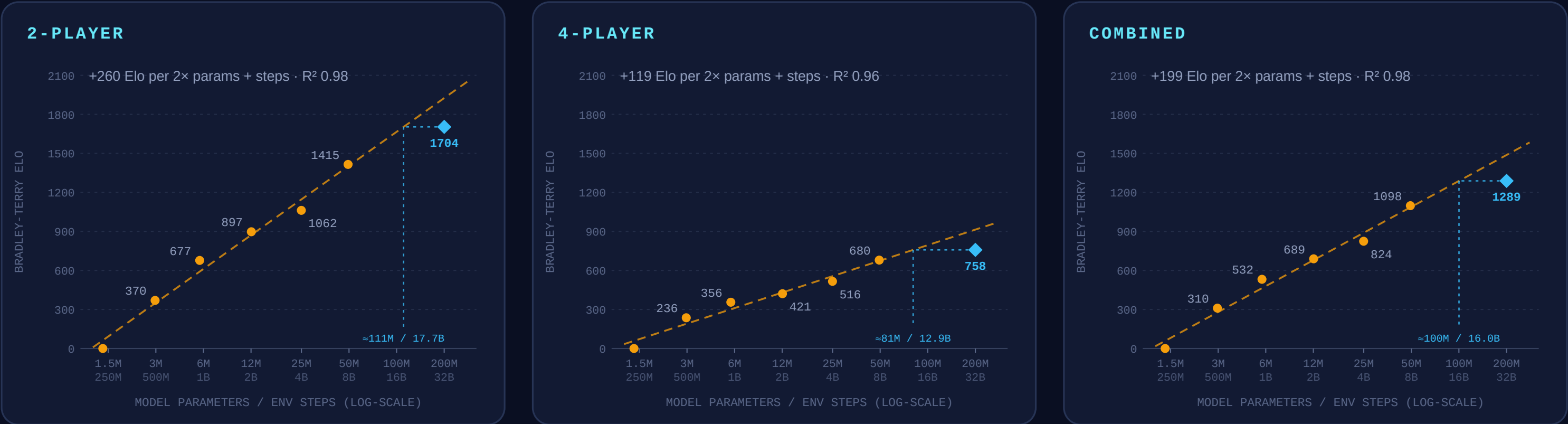
25M



200M

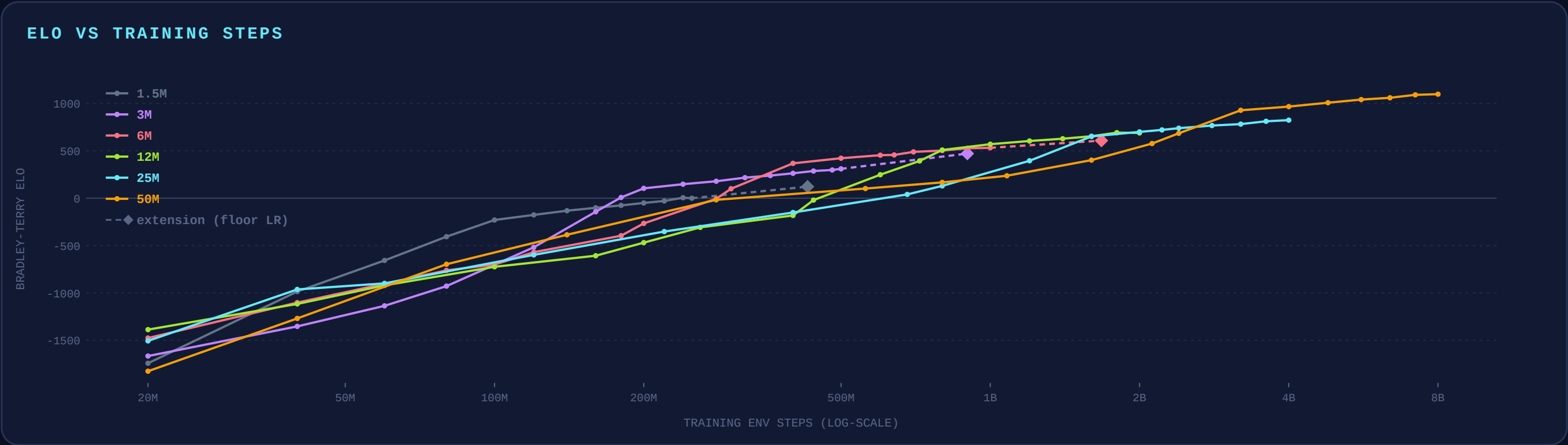
Scaling: Elo vs model size

- **Six controlled runs, 1.5M → 50M** — params and training steps double together per rung; log-linear Elo gains
- **Bradley-Terry Elo** from round-robin evals (4,096 games per pairing), anchored to the 1.5M model = 0
- **Elo is scaled log-odds**: a gap of d gives win odds of $10^{(d/400)} : 1$ — +100 = 1.78× the odds, +200 = 3.16× the odds, +400 = 10× the odds
- **Dashed fit: the six ladder runs only** — the 200M submitted run is placed on the same scale without refitting the line
- **Unexpectedly accurate**: on the joint ladder (200M ↔ 32B steps), the 200M's 14.7B steps correspond to a ≈92M-scale run — and the fit maps its combined Elo to ≈100M, despite warm restarts and LR / batch / mix changes en route
- **2p > 4p expected for 200M**: late training ran 90% 2p (ladder models: 75%)



Scaling: Elo over training

- **Same ladder scale, now over training** — combined (75 / 25 mix) Elo of each run's checkpoints, measured head-to-head
- **One S-curve, six times** — slow start, a sharp climb as cosine LR decay ends, then a steady floor-LR grind
- **No model plateaus** — the dashed extensions keep gaining +100–190 Elo per doubling of steps, entirely at floor LR
- **The compute-optimal size moves with budget** — roughly: 6M best for ~0.1–0.7B steps, 12M for ~0.8–2B, 25M for ~2–3B, 50M beyond



03

SECTION 03

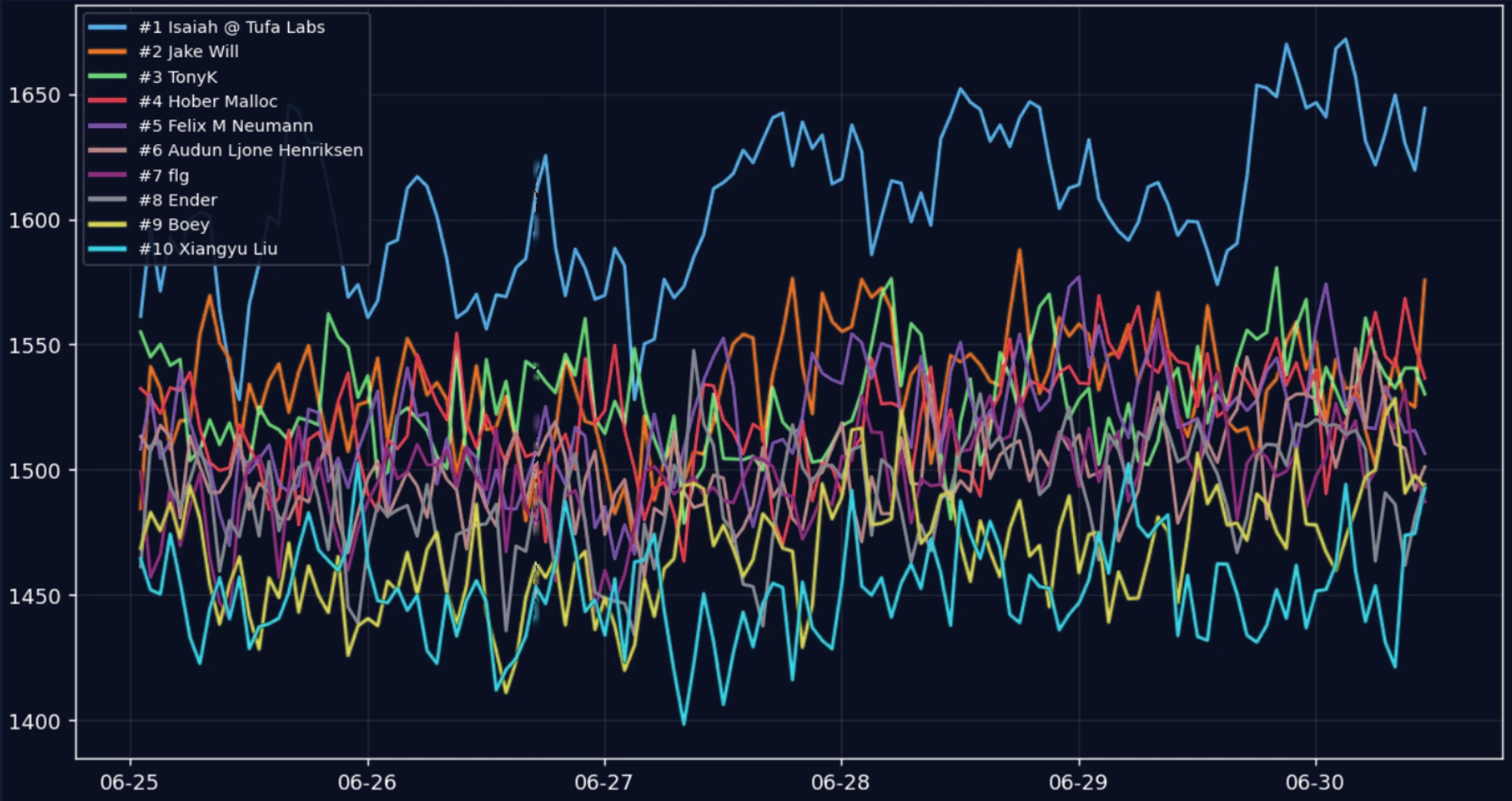
Flight recordings

Leaderboard standing, plus sample games.



Results: tentative 1st place

ELO OVER TIME — SINCE DEADLINE



WIN RATE — LIVE

2P **88.0%**

1985 / 2256 games
41.7% of games · baseline 50%

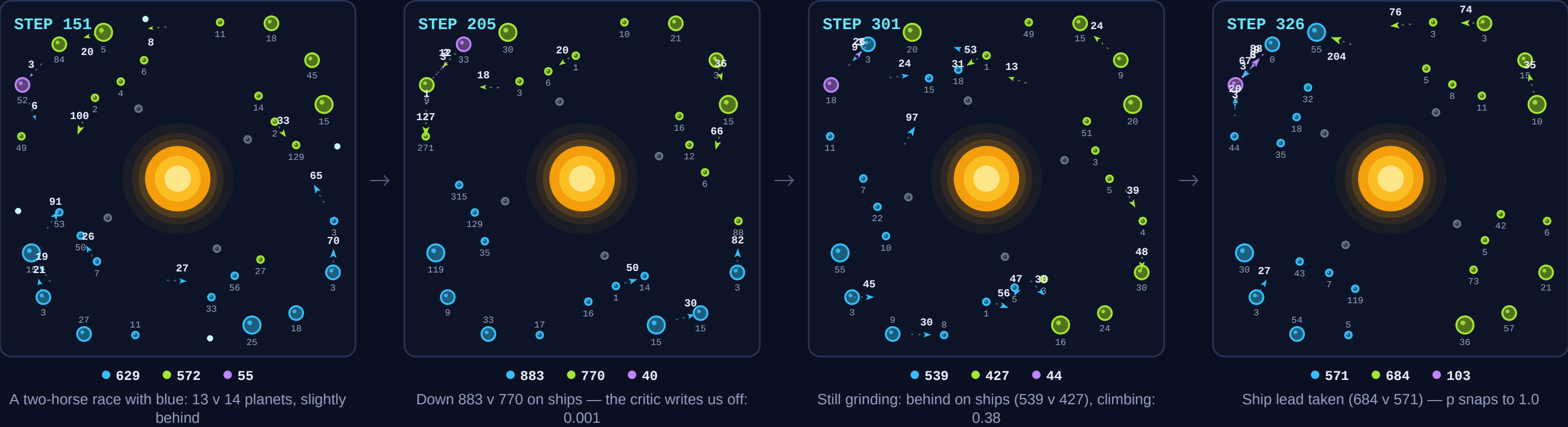
4P **29.4%**

927 / 3150 games
58.3% of games · baseline 25%

Live ratings are noisy — standings are final only when the leaderboard locks. 🙌

Game snapshot #1: tenacity

4-player game · lime = our agent · blue = the rival “Jake Will” · coral = Ender (dead), violet = TonyK (marginal)



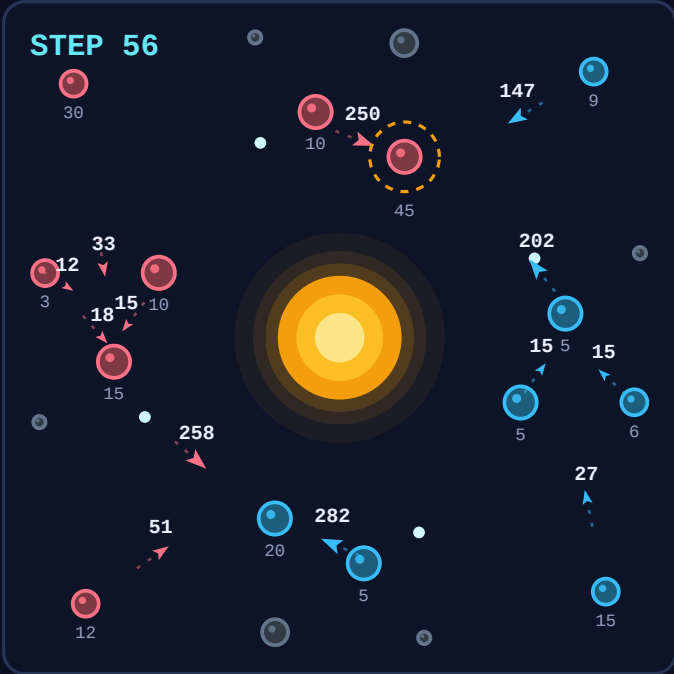
Game snapshot #2: opening tempo

2-player game · blue = our agent · coral = “Felix M Neumann” · no combat has occurred before step 55



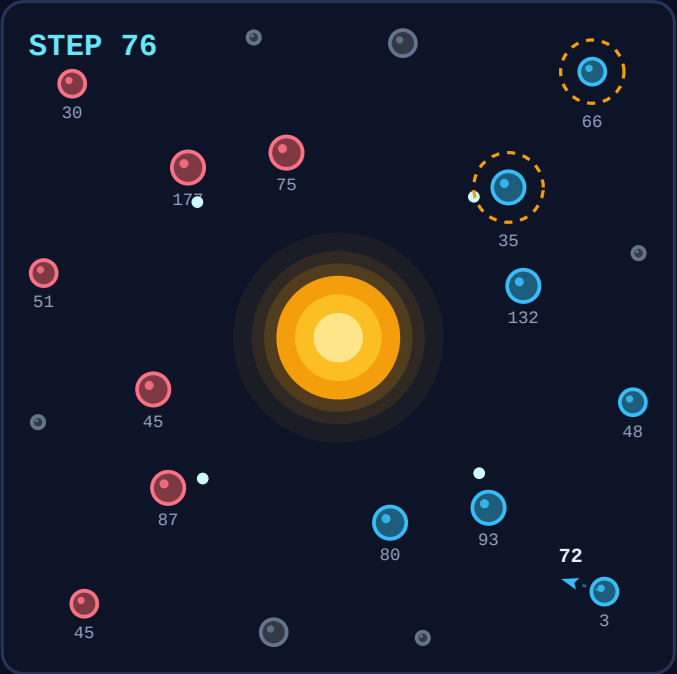
● 491 ● 500

Even production, down 9 ships — but our ships are better placed: 130 in flight



● 753 ● 762

Better placement converts: a 115-ship strike pays down their prod-5's garrison



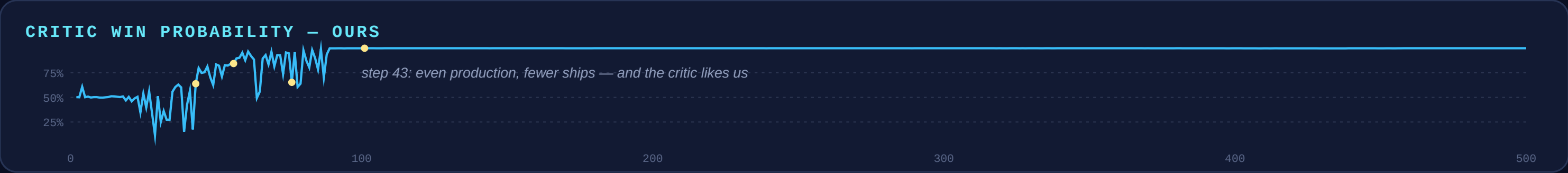
● 529 ● 510

The exchange resolves: their 13 & 17 captured, our planet 2 held — ship lead taken (529 v 510)



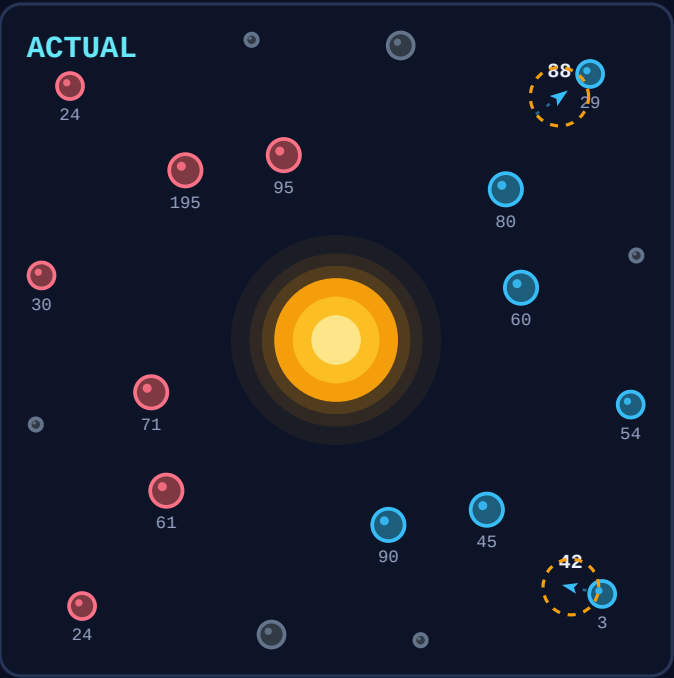
● 458 ● 379

Production 34 v 24 — the opponent never recovered



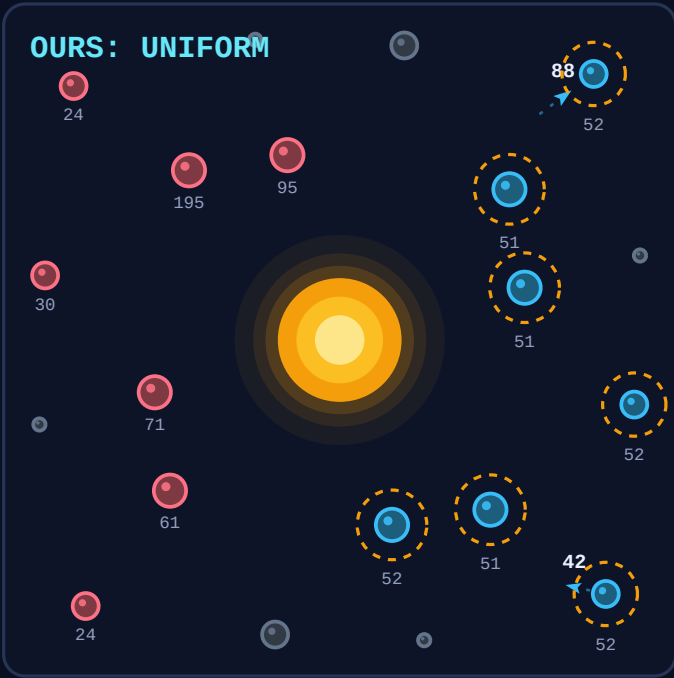
Game snapshot #2: interrogating the critic

Step 43, re-scored by the unquantized critic · identical ship totals in every frame (491 v 500) — only placement changes



p_win = 0.57

Flat garrisons + a 130-ship mobile mass — the real position



p_win = 0.31 Δ -0.26

Same ships spread evenly across our planets



p_win = 0.02 Δ -0.55

Same ships piled on our rearmost planet



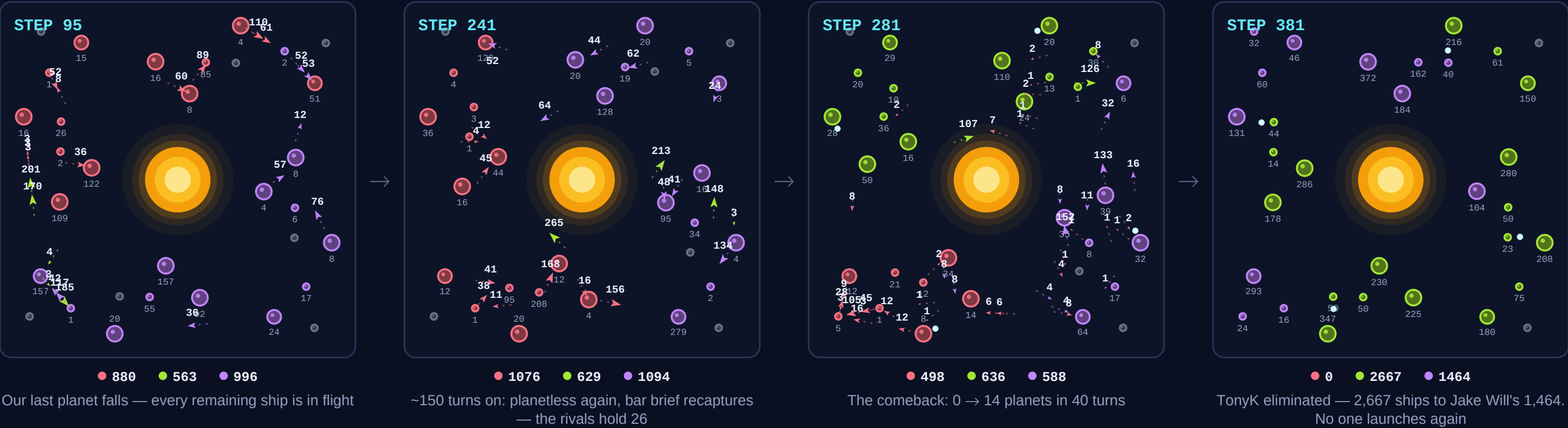
p_win = 0.17 Δ -0.40

Their ideal spread — easier said than done: no convenient way to rearrange mid-game

Ship placement alone moves the critic's win probability by more than 0.5.

Game snapshot #3: zero planets to victory

4-player game · lime = our agent · the dominant rivals: coral = “TonyK”, violet = “Jake Will”



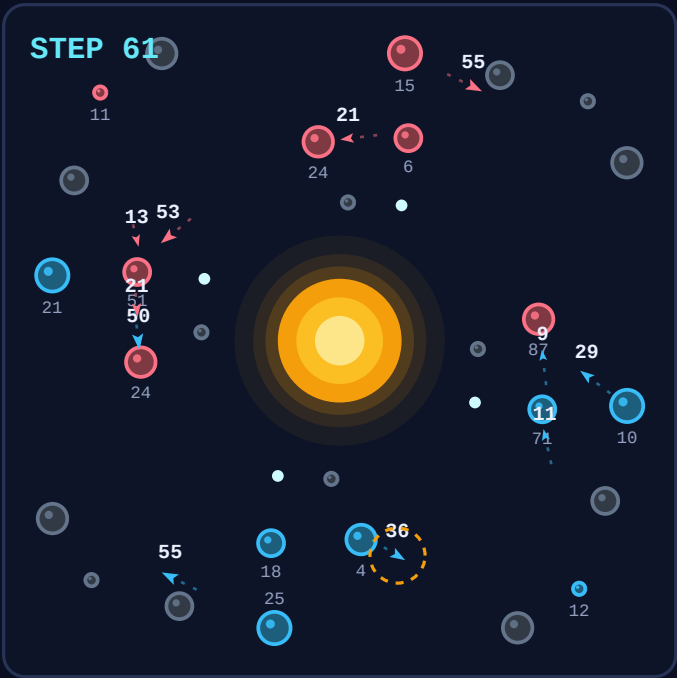
Game snapshot #4: learned internal lookahead

2-player game · blue = our agent · coral = “Felix M Neumann”



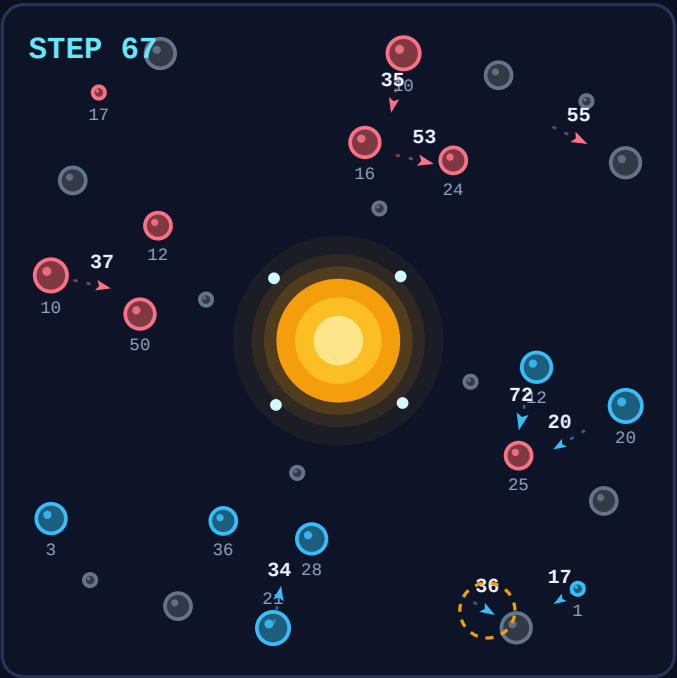
● 385 ● 393

12 ships launch east, reinforcing an orbiting neighbor



● 351 ● 381

The reinforced planet launches 36 southeast



● 300 ● 344

36 in flight across the southern map



● 299 ● 347

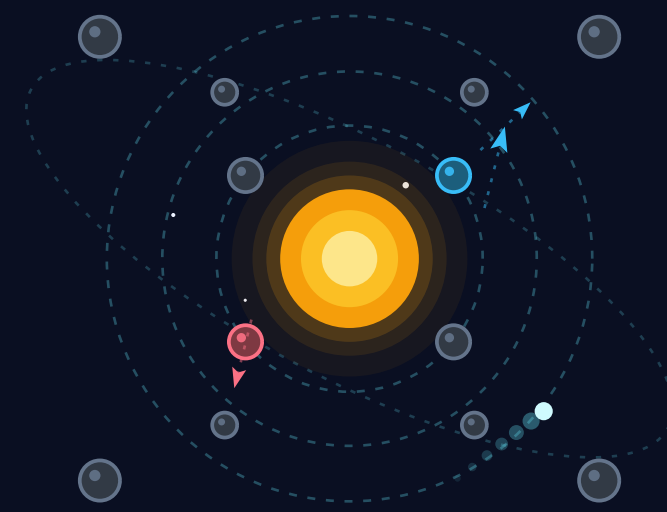
Target captured — 1 ship to spare

04

SECTION 04

Post-flight debrief

What I'd change next time.



What I'd do differently next time

STALLING

- $\gamma = 1.0$ keeps 4p win probabilities well-defined — but no urgency
- **Result:** grab a ship lead, then stall for the rest of the game
- **Wasteful in training** — decided games burn compute
- **Next time:** early truncation or surrender

2P VS 4P BALANCE

- **Matchmaking was 2p-heavy** → trained 90% 2p, skipped league play
- **Post-deadline it inverted** — now heavily 4-player
- **Next time:** balanced modes + league play vs past checkpoints

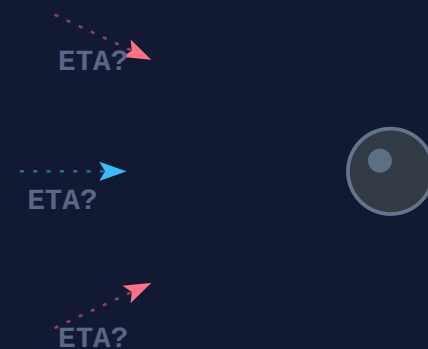
SCALING LAWS

- **Undersaturated:** the ladder suggests the 200M is $\sim 2\times$ short on training steps
- **Plateau or not?** I eyeballed it — disciplined scaling runs would have answered directly
- **Next time:** study scaling laws upfront, before the full-size run

Fleets as features, not entities

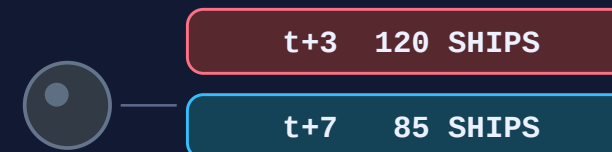
WHAT I BUILT

- **Every fleet is its own token** — target + ETA inferred from raw kinematics
- **Attention re-derives the physics** every single turn
- **Fleet cap at inference** — fleets beyond the cap are invisible to the model



WHAT SOME COMPETITORS DID

- **No fleet tokens** — fold fleets into target-planet features as ETA arrival schedules
- **“What arrives where, when?”** lands directly in the deciding token
- **Shorter sequences**, cheaper attention, no cap



TRANSMISSION ENDS

Thank you!

Questions?

